REVISION CHEAT SHEET:

LINQ in C#

LINQ / C#

Interview-ready notes · examples · C# · diagrams

Topic 24 of 40

LINQ stands for:

EXAMPLE / FLOW

```
Language Integrated Query
```

It gives C# a consistent way to:

EXAMPLE / FLOW

```
Filter

Transform

Sort

Group

Join

Aggregate

Search

Partition

Project
```

data.

Instead of manually writing loops such as:

C#

```
List<Gift> largeGifts = [];

foreach (Gift gift in gifts)
{
    if (gift.TotalAmount >= 1000m)
    {
        largeGifts.Add(gift);
    }
}
```

you can write:

C#

```
IEnumerable<Gift> largeGifts = gifts.Where(gift => gift.TotalAmount >= 1000m);
```

The important difference is not merely fewer lines of code.

LINQ lets you describe:

EXAMPLE / FLOW

```
WHAT data you want
```

LINQ / C#

rather than always spelling out:

EXAMPLE / FLOW

```
HOW to loop through the data
```

LINQ provides one consistent query model across in-memory objects and provider-backed sources such as Entity Framework Core, although the execution mechanism is very different between `IEnumerable<T>` and `IQueryable<T>`.

High-Level Mental Model

EXAMPLE / FLOW

```
Data Source
    ↓
LINQ Operators
    ↓
Query Description
    ↓
Execution
    ↓
Results
```

Example:

C#

```
IEnumerable<Gift> query =
    gifts
        .Where(
            gift =>
                gift.TotalAmount >= 1000m)
        .OrderByDescending(
            gift =>
                gift.TotalAmount)
        .Select(
            gift =>
                gift);
```

Flow:

EXAMPLE / FLOW

```
gifts
    ↓
Where
    ↓
OrderByDescending
    ↓
Select
    ↓
Result sequence
```

The Three Parts of a LINQ Query

Microsoft describes a typical LINQ query as having three stages:

EXAMPLE / FLOW

```
1. Obtain data source

2. Define query

3. Execute query
```

LINQ / C#

Defining the query and executing it are often separate operations because many LINQ operators use deferred execution.

Example:

C#

```
List<Gift> gifts = [new Gift(1, 100m), new Gift(2, 2500m), new Gift(3, 500m)];
```

Data source:

C#

```
List<Gift> gifts
```

Query definition:

C#

```
IEnumerable<Gift> query = gifts.Where(gift => gift.TotalAmount >= 1000m);
```

Execution:

C#

```
foreach (Gift gift in query)
{
    Console.WriteLine(
        gift.GiftId);
}
```

LINQ Is Built on Existing C# Features

LINQ depends heavily on concepts you already learned:

EXAMPLE / FLOW

```
Generics

IEnumerable<T>

Delegates

Func<T,...>

Lambda expressions

Extension methods

Anonymous types

Type inference with var
```

For example:

C#

```
gifts.Where(
    gift =>
        gift.TotalAmount > 100m);
```

LINQ / C#

`Where` is an extension method.

The lambda:

C#

```
gift =>
    gift.TotalAmount > 100m
```

represents filtering behavior.

For LINQ-to-Objects, it is based on a delegate shaped like:

C#

```
Func<Gift, bool>
```

1. LINQ to Objects

LINQ to Objects means applying LINQ operators to in-memory sources implementing:

C#

```
IEnumerable<T>
```

Examples:

EXAMPLE / FLOW

```
List<T>

T[]

HashSet<T>

Dictionary<TKey,TValue>

Queue<T>

Any custom IEnumerable<T>
```

Microsoft uses the term LINQ-to-Objects for queries directly over `IEnumerable<T>` sources.

Example

C#

```
List<decimal> amounts =
[
    100m,
    500m,
    2000m,
    50m
];

IEnumerable<decimal> largeAmounts = amounts.Where(amount => amount >= 500m);
```

This code executes against objects already available in .NET memory.

2. LINQ Method Syntax

Method syntax calls LINQ extension methods directly.

Example:

C#

```
IEnumerable<Gift> query =
    gifts
        .Where(
            gift =>
                gift.TotalAmount >= 1000m)
        .OrderBy(
            gift =>
                gift.GiftId);
```

This is sometimes called:

EXAMPLE / FLOW

```
Fluent syntax
```

because operators are chained together.

Microsoft's LINQ documentation notes that most query operations can be expressed directly through standard query operator method calls.

Method-Chaining Mental Model

EXAMPLE / FLOW

```
Source
    .Operator1(...)
    .Operator2(...)
    .Operator3(...)
```

Each operator returns something that can often become the source of the next operator.

Example:

C#

```
gifts
    .Where(...)
    .OrderBy(...)
    .Select(...)
```

3. LINQ Query Syntax

C# also provides SQL-like query syntax.

Example:

C#

```
IEnumerable<Gift> query =
    from gift in gifts
    where gift.TotalAmount >= 1000m
    orderby gift.GiftId
    select gift;
```

The compiler translates query syntax into standard LINQ method calls. Query syntax and method syntax therefore use the same underlying query model.

Query Syntax Structure

EXAMPLE / FLOW

```
from
    ↓
where
    ↓
orderby
    ↓
select
```

Example:

C#

```
var query =
    from gift in gifts
    where gift.Status == GiftStatus.Completed
    orderby gift.TotalAmount descending
    select gift;
```

SQL vs LINQ Order

SQL commonly looks like:

C#

```
SELECT ...
FROM ...
WHERE ...
ORDER BY ...
```

LINQ query syntax starts with:

C#

```
from
```

Example:

C#

```
from gift in gifts
where gift.TotalAmount > 100m
select gift;
```

Query Syntax Is Converted to Methods

This:

C#

```
var query =
    from gift in gifts
    where gift.TotalAmount > 1000m
    orderby gift.TotalAmount descending
    select gift;
```

is conceptually translated to:

C#

```
var query =
    gifts
```

```
        .Where(
            gift =>
                gift.TotalAmount > 1000m)
        .OrderByDescending(
            gift =>
                gift.TotalAmount)
        .Select(
            gift =>
                gift);
```

LINQ / C#

Microsoft explicitly documents this compiler translation.

Which Syntax Should You Use?

Both are valid.

Method syntax is especially common in modern application code because:

EXAMPLE / FLOW

```
Every LINQ operator is available

Chaining is explicit

Lambdas compose naturally

Most documentation uses method syntax
```

Query syntax can sometimes improve readability for:

EXAMPLE / FLOW

```
Complex joins

Multiple from clauses

Grouping

SQL-like queries
```

Some operations have no dedicated query-expression syntax and must be expressed through methods.

Mixing Both Styles

You can combine them.

Example:

C#

```
int count =
    (
        from gift in gifts
        where gift.TotalAmount > 1000m
        select gift
    )
    .Count();
```

This is perfectly valid.

4. Where — Filtering

`Where` keeps elements satisfying a condition.

```
C#
IEnumerable<Gift> completed = gifts.Where(gift => gift.Status == GiftStatus.Completed
```

Conceptually:

```
EXAMPLE / FLOW
For each element
    ↓
Predicate returns true?
    ├─ Yes → keep
    └─ No  → skip
```

Multiple Conditions

```
C#
IEnumerable<Gift> query =
    gifts.Where(
        gift =>
            gift.Status
                == GiftStatus.Completed && gift.TotalAmount >= 1000m);
```

Chained Where

This:

```
C#
var query =
    gifts
        .Where(
            gift =>
                gift.Status
                ==
                GiftStatus.Completed)
        .Where(
            gift =>
                gift.TotalAmount
                >= 1000m);
```

is also valid.

Whether combining conditions or chaining them is clearer depends on the query.

Dynamic Query Composition

LINQ makes conditional filtering convenient.

```
C#
IEnumerable<Gift> query = gifts;

if (
    minimumAmount is not null)
{
    query = query.Where(gift => gift.TotalAmount >= minimumAmount.Value);
}

if (
    status is not null)
{
```

```
    query = query.Where(gift => gift.Status == status);
}
```

LINQ / C#

This idea becomes extremely useful with:

```
IQueryable<Gift>
```

in database queries.

5. Select — Projection

`Select` transforms each element into another shape.

Source:

```
Gift
```

Result:

```
decimal
```

Example:

```
IEnumerable<decimal> amounts = gifts.Select(gift => gift.TotalAmount);
```

Flow:

```
Gift
    ↓
Select
    ↓
decimal
```

Projection to DTO

```
IEnumerable<GiftSummary> summaries =
    gifts.Select(
        gift =>
            new GiftSummary(
                gift.GiftId,
                gift.TotalAmount));
```

This is a major LINQ use case.

Projection to Anonymous Type

```
var summaries =
    gifts.Select(
        gift =>
            new
            {
```

```
            gift.GiftId,
            gift.TotalAmount
        });
```

LINQ / C#

The result element is an anonymous type.

Query Syntax Projection

```C#
var summaries =
    from gift in gifts
    select new
    {
        gift.GiftId,
        gift.TotalAmount
    };
```

Select Does Not Mutate Source

Suppose:

```C#
IEnumerable<decimal> amounts = gifts.Select(gift => gift.TotalAmount);
```

The original:

```C#
List<Gift> gifts
```

has not been changed.

LINQ usually creates:

```EXAMPLE / FLOW
A new result sequence
```

rather than modifying the source collection.

6. SelectMany — Flattening

Suppose every Gift has:

```C#
IReadOnlyList<GiftAllocation>
```

Then:

```C#
IEnumerable<IEnumerable<GiftAllocation>>
```

would represent:

```EXAMPLE / FLOW
Sequence of allocation sequences
```

But often you want:

EXAMPLE / FLOW

```
One flat sequence
```

LINQ / C#

Use:

C#

```
SelectMany
```

Example

C#

```
IEnumerable<GiftAllocation> allocations = gifts.SelectMany(gift => gift.Allocations);
```

Flow:

EXAMPLE / FLOW

```
Gift 1
    → Allocation A
    → Allocation B

Gift 2
    → Allocation C

Gift 3
    → Allocation D
    → Allocation E
```

`SelectMany` produces:

EXAMPLE / FLOW

```
A
B
C
D
E
```

Select vs SelectMany

`Select`:

EXAMPLE / FLOW

```
1 element
    →
1 projected result
```

`SelectMany`:

EXAMPLE / FLOW

```
1 element
    →
0..N child results
    ↓
Flatten everything
```

Example

```C#
var nested = gifts.Select(gift => gift.Allocations);
```

Result conceptually:

```EXAMPLE / FLOW
IEnumerable<IReadOnlyList<GiftAllocation>>
```

Flattened

```C#
var flat = gifts.SelectMany(gift => gift.Allocations);
```

Result:

```EXAMPLE / FLOW
IEnumerable<GiftAllocation>
```

7. Ordering

Use:

```C#
OrderBy
```

for ascending order.

```C#
IEnumerable<Gift> ordered = gifts.OrderBy(gift => gift.TotalAmount);
```

Descending

```C#
IEnumerable<Gift> ordered = gifts.OrderByDescending(gift => gift.TotalAmount);
```

Multiple Sort Keys

Use:

```C#
ThenBy
```

and:

```C#
ThenByDescending
```

Example:

```C#
IEnumerable<Gift> ordered =
    gifts
        .OrderBy(
            gift =>
                gift.Status)
        .ThenByDescending(
            gift =>
                gift.TotalAmount)
        .ThenBy(
            gift =>
                gift.GiftId);
```

Important Mistake

Don't write:

```C#
gifts
    .OrderBy(
        gift =>
            gift.Status)
    .OrderBy(
        gift =>
            gift.TotalAmount);
```

The second:

```C#
OrderBy
```

starts a new primary ordering.

If you want a secondary key:

```C#
ThenBy
```

is the correct operation.

Query Syntax

```C#
var query = from gift in gifts orderby gift.Status, gift.TotalAmount descending select gift;
```

Sorting Is Deferred but Non-Streaming

`OrderBy` doesn't normally execute when the query is defined.

However, once enumeration begins, LINQ-to-Objects must obtain enough source information to sort it before yielding ordered results.

Therefore sorting is:

```EXAMPLE / FLOW
Deferred
+
Non-streaming
```

Microsoft classifies ordering/grouping-style operations as deferred operations that may require reading all source data before yielding results.

8. Deferred Execution

One of the most important LINQ concepts is:

EXAMPLE / FLOW

```
Query definition
≠
Query execution
```

Example:

C#

```
List<int> numbers = [1, 2, 3];

IEnumerable<int> query = numbers.Where(number => number > 1);
```

At this moment:

EXAMPLE / FLOW

```
Where has not necessarily enumerated numbers.
```

Now add:

C#

```
numbers.Add(
    4);
```

Enumerate:

C#

```
foreach (
    int number
    in query)
{
    Console.WriteLine(
        number);
}
```

Result:

EXAMPLE / FLOW

```
2
3
4
```

because the source is evaluated when the deferred query is enumerated. Microsoft explicitly notes that deferred query results depend on source contents at execution time, not necessarily at definition time.

Deferred Execution Mental Model

EXAMPLE / FLOW

```
Define recipe
    ↓
Nothing enumerated yet
    ↓
```

```
Source changes
      ↓
Enumerate recipe
      ↓
Current source evaluated
```

LINQ / C#

Operators Commonly Using Deferred Execution

Examples include:

EXAMPLE / FLOW

```
Where

Select

SelectMany

OrderBy

GroupBy

Distinct

Skip

Take

Concat
```

The exact streaming behavior differs among operators.

9. Immediate Execution

Some operators must produce a result immediately.

Examples:

EXAMPLE / FLOW

```
Count

Any

All

First

Single

Sum

Average

Min

Max
```

Example:

C#

```
int count = gifts.Count();
```

The method must determine the answer before returning the integer.

Microsoft classifies scalar-producing operators such as `Count`, `First`, `Max`, and `Average` as immediately executing operations.

LINQ / C#

Materialization Also Forces Execution

Methods such as:

```
C#
ToList()

ToArray()

ToDictionary()

ToLookup()
```

materialize results into a data structure.

Example:

```
C#
List<Gift> largeGifts = gifts.Where(gift => gift.TotalAmount >= 1000m).ToList();
```

Now:

```
EXAMPLE / FLOW
Query has executed
```

and the resulting values are stored in:

```
C#
largeGifts
```

Microsoft describes `ToList` and `ToArray` as mechanisms for forcing execution and caching query results.

Deferred vs Materialized

Deferred:

```
C#
IEnumerable<Gift> query = gifts.Where(...);
```

Materialized:

```
C#
List<Gift> results = gifts.Where(...).ToList();
```

When Materialization Is Useful

Use it intentionally when:

```
EXAMPLE / FLOW
You need a snapshot

You need repeated access without re-querying

You need List<T> operations
```

```
You need to cross a disposed resource boundary

You need to prevent repeated database execution
```

Do Not Materialize Too Early

Bad database-style pattern:

C#

```
List<Gift> gifts = await dbContext.Gifts.ToListAsync();

List<Gift> filtered = gifts.Where(gift => gift.TotalAmount >= 1000m).ToList();
```

This may load:

EXAMPLE / FLOW

```
Every Gift
```

into memory before filtering.

Better:

C#

```
List<Gift> filtered =
    await dbContext.Gifts
        .Where(
            gift =>
                gift.TotalAmount >= 1000m)
        .ToListAsync();
```

Now the provider can potentially apply filtering in the database.

EF Core guidance specifically recommends composing the query before materialization, projecting only needed data, limiting result sets, and avoiding unnecessary buffering.

10. Repeated Enumeration

Consider:

C#

```
IEnumerable<Gift> query = GetGifts().Where(gift => gift.TotalAmount >= 1000m);
```

Then:

C#

```
int count = query.Count();

decimal total = query.Sum(gift => gift.TotalAmount);
```

If `query` is deferred, the source may be enumerated:

EXAMPLE / FLOW

```
Once for Count

Again for Sum
```

In-Memory Source

This might merely mean:

```
EXAMPLE / FLOW
Two loops
```

which could be fine.

Database-backed Source

If it remains:

```
C#
IQueryable<Gift>
```

it might mean:

```
EXAMPLE / FLOW
Two database queries
```

Therefore:

```
EXAMPLE / FLOW
Understand what your source actually is.
```

Materialize Intentionally

If you truly need the same result set multiple times:

```
C#
List<Gift> results = await query.ToListAsync();
```

Then:

```
C#
int count = results.Count;

decimal total = results.Sum(gift => gift.TotalAmount);
```

But only do this when loading those results into memory is appropriate.

11. Streaming vs Non-Streaming LINQ

Deferred execution can be divided conceptually into:

```
EXAMPLE / FLOW
Streaming

Non-streaming
```

Microsoft makes this distinction in its standard query operator execution classification.

Streaming Operator

Example:

C#

```
Where
```

can inspect one item and potentially yield it immediately.

EXAMPLE / FLOW

```
Read Gift 1
    ↓
Matches?
    ↓
Yield Gift 1
```

It doesn't necessarily need the entire source first.

Another Streaming Operator

C#

```
Select
```

can transform each item as it is requested.

Non-Streaming Operator

`OrderBy` must generally know the source values before it can determine sorted order.

EXAMPLE / FLOW

```
Read all
    ↓
Sort
    ↓
Yield first ordered result
```

GroupBy

`GroupBy` also needs to organize source elements into groups before producing the grouped representation.

Why This Matters

For:

EXAMPLE / FLOW

```
10 elements
```

usually irrelevant.

For:

EXAMPLE / FLOW

```
10 million elements
```

it can significantly affect:

EXAMPLE / FLOW

```
Memory

Latency to first result
```

```
Overall performance
```

12. Any

Use `Any` to ask:

EXAMPLE / FLOW

```
Does at least one element exist?
```

Example:

C#

```
bool hasCompletedGifts = gifts.Any(gift => gift.Status == GiftStatus.Completed);
```

Any Without Predicate

C#

```
bool hasAny = gifts.Any();
```

Prefer Any for Existence

Instead of:

C#

```
if (
    gifts.Count()
    >
    0)
```

prefer:

C#

```
if (
    gifts.Any())
```

because `Any` expresses the intent directly and can stop once existence is proven.

With provider-backed sources, it can also translate to an efficient existence query where supported.

13. All

`All` asks:

EXAMPLE / FLOW

```
Does every element satisfy this condition?
```

Example:

C#

```
bool allValid = gifts.All(gift => gift.TotalAmount > 0m);
```

Important Empty-Sequence Rule

For an empty sequence:

```C#
gifts.All(
    predicate)
```

returns:

```
EXAMPLE / FLOW
true
```

because there is no element violating the predicate.

This is sometimes called:

```
EXAMPLE / FLOW
Vacuous truth
```

If You Require At Least One

Use:

```C#
bool allValidAndNonEmpty = gifts.Any() && gifts.All(gift => gift.TotalAmount > 0m);
```

14. Contains

```C#
bool contains = giftIds.Contains(giftId);
```

For LINQ-to-Objects, equality semantics depend on the element's equality comparer.

For database providers, translation depends on the provider and expression shape.

Frequent In-Memory Membership

If repeatedly doing:

```C#
allowedIds.Contains(
    giftId);
```

against a large in-memory sequence, a:

```C#
HashSet<long>
```

may be more appropriate than repeatedly scanning a list.

That connects Topic 21 collections to LINQ performance.

15. First

```C#
Gift gift = gifts.First();
```

Returns the first element.

If sequence is empty:

EXAMPLE / FLOW

```
InvalidOperationException
```

LINQ / C#

First with Predicate

C#

```
Gift gift = gifts.First(gift => gift.Status == GiftStatus.Completed);
```

Returns the first matching element.

Throws when no matching element exists.

FirstOrDefault

C#

```
Gift? gift = gifts.FirstOrDefault();
```

If none:

EXAMPLE / FLOW

```
default(Gift)
    →
null
```

for a reference type.

Predicate

C#

```
Gift? gift = gifts.FirstOrDefault(gift => gift.GiftId == targetId);
```

16. Single

`Single` communicates:

There must be exactly one matching element.

C#

```
Gift gift = gifts.Single(gift => gift.GiftId == giftId);
```

Behavior:

EXAMPLE / FLOW

```
0 matches
    →
Exception

1 match
    →
Return item

2+ matches
    →
Exception
```

SingleOrDefault

C#
```
Gift? gift = gifts.SingleOrDefault(gift => gift.GiftId == giftId);
```

Behavior:

EXAMPLE / FLOW
```
0
    →
default

1
    →
item

2+
    →
Exception
```

First vs Single

`First` says:

EXAMPLE / FLOW
```
Give me one,
specifically the first one.
```

`Single` says:

EXAMPLE / FLOW
```
There must be exactly one.
```

Use `Single` when uniqueness is genuinely part of the invariant.

Use `First` when multiple values are acceptable but you need one according to sequence/order semantics.

FirstOrDefault vs SingleOrDefault

Situation	FirstOrDefault	SingleOrDefault
0 matches	default	default
1 match	value	value
2+ matches	first match	exception

17. Last and LastOrDefault

C#
```
Gift gift = gifts.Last();
```

or:

C#
```
Gift? gift = gifts.LastOrDefault();
```

Be aware that what “last” means depends on source ordering.

In database queries:

EXAMPLE / FLOW

```
No explicit ordering
```

generally means you should not rely on a meaningful business “last row.”

Use:

C#

```
OrderBy(...)
```

before choosing a first/last element when business order matters.

18. Aggregation

LINQ supports aggregate operators such as:

EXAMPLE / FLOW

```
Count

LongCount

Sum

Average

Min

Max

MinBy

MaxBy

Aggregate
```

Count

C#

```
int completedCount = gifts.Count(gift => gift.Status == GiftStatus.Completed);
```

Sum

C#

```
decimal total = gifts.Sum(gift => gift.TotalAmount);
```

Average

C#

```
decimal average = gifts.Average(gift => gift.TotalAmount);
```

Be careful with empty sequences depending on the overload/type.

Min

```C#
decimal smallest = gifts.Min(gift => gift.TotalAmount);
```

Max

```C#
decimal largest = gifts.Max(gift => gift.TotalAmount);
```

MaxBy

```C#
Gift? largestGift = gifts.MaxBy(gift => gift.TotalAmount);
```

This returns the element associated with the maximum key rather than only the maximum key itself.

MinBy

```C#
Gift? smallestGift = gifts.MinBy(gift => gift.TotalAmount);
```

19. Aggregate

`Aggregate` allows custom accumulation.

Example:

```C#
decimal total =
    gifts.Aggregate(
        0m,
        (
            runningTotal,
            gift) =>
            runningTotal
            +
            gift.TotalAmount);
```

Flow:

```EXAMPLE / FLOW
0
    +
Gift 1 amount
    ↓
running total
    +
Gift 2 amount
    ↓
...
```

For simple totals:

C#

```
Sum
```

LINQ / C#

is clearer.

Use `Aggregate` when the accumulation logic is genuinely custom.

20. Grouping with GroupBy

Suppose you want gifts by Campaign.

C#

```
IEnumerable<IGrouping<long?, Gift>>
    groups = gifts.GroupBy(gift => gift.CampaignId);
```

Each group contains:

EXAMPLE / FLOW

```
Key
+
Elements
```

Iterate Groups

C#

```
foreach (
    IGrouping<long?, Gift> group
    in groups)
{
    Console.WriteLine(
        $"Campaign: {group.Key}");

    foreach (
        Gift gift
        in group)
    {
        Console.WriteLine(
            gift.GiftId);
    }
}
```

GroupBy Mental Model

Input:

EXAMPLE / FLOW

```
Gift 1 → Campaign 10
Gift 2 → Campaign 20
Gift 3 → Campaign 10
Gift 4 → Campaign 20
Gift 5 → Campaign 10
```

Output:

EXAMPLE / FLOW

```
Campaign 10
    Gift 1
    Gift 3
```

```
    Gift 5
Campaign 20
    Gift 2
    Gift 4
```

LINQ / C#

Group + Aggregate

```C#
var totalsByCampaign =
    gifts
        .GroupBy(
            gift =>
                gift.CampaignId)
        .Select(
            group =>
                new
                {
                    CampaignId =
                        group.Key,

                    GiftCount =
                        group.Count(),

                    TotalAmount = group.Sum(gift => gift.TotalAmount) });
```

This is a very common reporting pattern.

Query Syntax

```C#
var totalsByCampaign =
    from gift in gifts
    group gift
        by gift.CampaignId
        into giftGroup
    select new
    {
        CampaignId =
            giftGroup.Key,

        GiftCount =
            giftGroup.Count(),

        TotalAmount = giftGroup.Sum(gift => gift.TotalAmount) };
```

21. ToLookup

`ToLookup` also groups values by key but immediately creates an:

```C#
ILookup<TKey,TElement>
```

Example:

```C#
ILookup<long?, Gift> lookup = gifts.ToLookup(gift => gift.CampaignId);
```

Now:

C#

```
IEnumerable<Gift> campaignGifts = lookup[campaignId];
```

LINQ / C#

GroupBy vs ToLookup

`GroupBy`:

EXAMPLE / FLOW

```
Query operator

Deferred result

Produces IGrouping sequences
```

`ToLookup`:

EXAMPLE / FLOW

```
Immediate materialization

Creates reusable lookup structure

Lookup by key
```

Use `ToLookup` when you deliberately want an in-memory grouped lookup.

22. Modern CountBy

.NET 9 introduced:

C#

```
CountBy
```

which remains available in .NET 10.

It counts elements by key without requiring an intermediate `GroupBy` allocation pattern.

Example:

C#

```
IEnumerable<KeyValuePair<GiftStatus, int>>
    counts = gifts.CountBy(gift => gift.Status);
```

Conceptually:

EXAMPLE / FLOW

```
Pending    → 10
Completed  → 25
Cancelled  → 3
```

Old GroupBy Version

C#

```
var counts =
    gifts
        .GroupBy(
            gift =>
                gift.Status)
        .Select(
```

```
            group =>
                new
                {
                    Status =
                        group.Key,

                    Count = group.Count() });
```

LINQ / C#

`CountBy` can express frequency counting more directly.

23. Modern AggregateBy

.NET 9 also added:

```
C#
AggregateBy
```

to aggregate state independently for each key without first materializing group objects.

Example:

```
C#
var totals =
    gifts.AggregateBy(
        gift =>
            gift.CampaignId,
        seed:
            0m,
        (
            total,
            gift) =>
            total
            +
            gift.TotalAmount);
```

Conceptually:

```
EXAMPLE / FLOW
Campaign 1 → total amount

Campaign 2 → total amount

Campaign 3 → total amount
```

For ordinary code, `GroupBy` remains excellent and readable.

`AggregateBy` becomes useful when per-key accumulation better describes the algorithm.

24. Joining Sequences

LINQ provides relational-style joining between sequences.

Suppose:

```
C#
List<Gift> gifts;
List<Campaign> campaigns;
```

Each Gift has:

```
C#
CampaignId
```

and each Campaign has:

```
C#
CampaignId
```

Inner Join

```
C#
var result =
    gifts.Join(
        campaigns,
        gift =>
            gift.CampaignId,
        campaign =>
            campaign.CampaignId,
        (
            gift,
            campaign) =>
            new
            {
                gift.GiftId,
                CampaignName = campaign.Name });
```

Microsoft describes `Join` as an equality-based inner join: only matching elements from both sequences appear.

Query Syntax Join

```
C#
var result =
    from gift in gifts
    join campaign in campaigns
        on gift.CampaignId
        equals campaign.CampaignId
    select new
    {
        gift.GiftId,
        CampaignName = campaign.Name };
```

Notice the keyword:

```
C#
equals
```

rather than:

```
C#
==
```

inside the query-syntax join clause.

Join Flow

EXAMPLE / FLOW

```
Gift.CampaignId
        ↓
match
        ↓
Campaign.CampaignId
        ↓
Combined result
```

25. GroupJoin

`GroupJoin` correlates each outer element with a collection of matching inner elements.

Example conceptual relationship:

EXAMPLE / FLOW

```
Campaign
    ↓
All Gifts belonging to Campaign
```

Example

C#

```
var result =
    campaigns.GroupJoin(
        gifts,
        campaign =>
            campaign.CampaignId,
        gift =>
            gift.CampaignId,
        (
            campaign,
            campaignGifts) =>
            new
            {
                campaign.Name,
                Gifts = campaignGifts });
```

Microsoft notes that `GroupJoin` creates hierarchical grouped matches and can serve as part of inner-join and outer-join patterns.

Query Syntax GroupJoin

C#

```
var result =
    from campaign in campaigns
    join gift in gifts
        on campaign.CampaignId
        equals gift.CampaignId
        into campaignGifts
    select new
    {
        campaign.Name,
        Gifts = campaignGifts };
```

26. Left Join in .NET 10

LINQ / C#

Before .NET 10, LINQ-to-Objects commonly expressed a left join using:

```
EXAMPLE / FLOW
GroupJoin
+
DefaultIfEmpty
+
SelectMany
```

.NET 10 added:

```
C#
LeftJoin
```

and:

```
C#
RightJoin
```

to `Enumerable` and `Queryable`.

LeftJoin Example

Conceptually:

```
C#
var result =
    campaigns.LeftJoin(
        gifts,
        campaign =>
            campaign.CampaignId,
        gift =>
            gift.CampaignId,
        (
            campaign,
            gift) =>
            new
            {
                campaign.Name,
                GiftId = gift?.GiftId });
```

Semantics:

```
EXAMPLE / FLOW
Every Campaign appears

Matching Gift?
    ├─ Yes → include Gift
    └─ No  → Gift side is empty/default
```

Provider support and SQL translation should still be verified for your specific query provider.

RightJoin

The opposite orientation:

EXAMPLE / FLOW

```
Every element from right sequence appears
```

LINQ / C#

with matching left-side elements when present.

This is one of the notable LINQ improvements available in .NET 10.

27. DefaultIfEmpty

`DefaultIfEmpty` returns the source when elements exist.

If the source is empty, it returns a sequence containing:

C#

```
default(T)
```

This was historically important for constructing left outer joins.

Example:

C#

```
IEnumerable<int> numbers = [];

IEnumerable<int> result = numbers.DefaultIfEmpty();
```

Result:

EXAMPLE / FLOW

```
0
```

because:

C#

```
default(int)
```

is `0`.

28. Set Operations

LINQ provides:

EXAMPLE / FLOW

```
Distinct

DistinctBy

Union

UnionBy

Intersect

IntersectBy

Except

ExceptBy
```

Microsoft classifies these as LINQ set operations based on element/key equality.

Distinct


```C#
IEnumerable<long> uniqueFundIds = fundIds.Distinct();
```

Input:

```EXAMPLE / FLOW
10
20
10
30
20
```

Result:

```EXAMPLE / FLOW
10
20
30
```

DistinctBy

Suppose you have:

```C#
IEnumerable<Gift> gifts
```

and want one gift for each constituent:

```C#
IEnumerable<Gift> onePerConstituent = gifts.DistinctBy(gift => gift.ConstituentId);
```

`DistinctBy` selects uniqueness using a key selector.

Union

```C#
IEnumerable<long> allIds = firstIds.Union(secondIds);
```

Returns unique elements present in either sequence.

Intersect

```C#
IEnumerable<long> sharedIds = firstIds.Intersect(secondIds);
```

Returns elements present in both.

Except

```C#
IEnumerable<long> missingIds = expectedIds.Except(processedIds);
```

Meaning:

EXAMPLE / FLOW

```
Expected
minus
Processed
```

LINQ / C#

Concat vs Union

`Concat`:

C#

```
first.Concat(
    second)
```

keeps sequence contents, including duplicates.

`Union`:

C#

```
first.Union(
    second)
```

performs set-style uniqueness.

Example

First:

EXAMPLE / FLOW

```
1 2 2
```

Second:

EXAMPLE / FLOW

```
2 3
```

`Concat`:

EXAMPLE / FLOW

```
1 2 2 2 3
```

`Union`:

EXAMPLE / FLOW

```
1 2 3
```

29. Partitioning

Useful methods include:

EXAMPLE / FLOW

```
Take

Skip

TakeWhile
```

```
SkipWhile
*
TakeLast

SkipLast
```

LINQ / C#

Take

C#

```
IEnumerable<Gift> firstTen = gifts.Take(10);
```

Skip

C#

```
IEnumerable<Gift> afterFirstTen = gifts.Skip(10);
```

Pagination Pattern

C#

```
int pageNumber = 3;

int pageSize = 25;

IEnumerable<Gift> page =
    gifts
        .Skip(
            (pageNumber - 1)
            *
            pageSize)
        .Take(
            pageSize);
```

Always Order Database Pagination

For database paging, prefer:

C#

```
query
    .OrderBy(
        gift =>
            gift.GiftId)
    .Skip(...)
    .Take(...)
```

rather than paging over an unordered result.

Ordering must be deterministic for stable pagination.

Offset Pagination Caveat

EF Core documentation notes that deep `Skip` / `Take` pagination has disadvantages:

EXAMPLE / FLOW

```
Database still processes skipped rows

Concurrent inserts/deletes can shift pages
```

For next/previous style navigation over large datasets, keyset/seek pagination can be more efficient and stable.

Keyset Concept

Instead of:

```C#
.Skip(
    100_000)
.Take(
    50)
```

you might use:

```C#
.Where(
    gift =>
        gift.GiftId
        >
        lastGiftId)
.Take(
    50)
```

with suitable indexing and deterministic ordering.

30. TakeWhile

```C#
IEnumerable<Gift> result = orderedGifts.TakeWhile(gift => gift.TotalAmount < 1000m);
```

Stops as soon as predicate becomes false.

This is sequence-position behavior, not the same as `Where`.

Where vs TakeWhile

Input amounts:

```EXAMPLE / FLOW
100
200
1500
300
```

`Where(amount < 1000)`:

```EXAMPLE / FLOW
100
200
300
```

`TakeWhile(amount < 1000)`:

```EXAMPLE / FLOW
100
200
```

because it stops after `1500`.

31. SkipWhile

Opposite idea.

```C#
IEnumerable<Gift> result = orderedGifts.SkipWhile(gift => gift.TotalAmount < 1000m);
```

Once predicate becomes false, remaining elements are returned.

32. Chunk

For batch processing:

```C#
IEnumerable<Gift[]> batches = gifts.Chunk(100);
```

Conceptually:

EXAMPLE / FLOW

```
Batch 1 → 100 gifts

Batch 2 → 100 gifts

Batch 3 → remaining gifts
```

Useful for in-memory batch operations.

Do not assume arbitrary database providers translate every modern LINQ operator.

33. Append and Prepend

```C#
IEnumerable<Gift> withExtra = gifts.Append(gift);
```

and:

```C#
IEnumerable<Gift> withFirst = gifts.Prepend(gift);
```

These return new enumerable sequences.

They do not mutate the original collection.

34. Zip

`Zip` combines sequences position by position.

```C#
string[] names = ["A", "B", "C"];

decimal[] amounts = [100m, 200m, 300m];

var zipped = names.Zip(amounts);
```

Conceptually:

EXAMPLE / FLOW

```
A ↔ 100

B ↔ 200

C ↔ 300
```

LINQ / C#

Use `Join` when matching by keys.

Use `Zip` when matching by sequence position.

35. OfType<T>

Filters elements by runtime type.

C#

```
IEnumerable<Gift> gifts = entities.OfType<Gift>();
```

Only elements that are actually compatible with:

C#

```
Gift
```

remain.

Cast<T>

C#

```
IEnumerable<Gift> gifts = entities.Cast<Gift>();
```

This attempts to cast every element.

If an element cannot be cast:

EXAMPLE / FLOW

```
InvalidCastException
```

OfType vs Cast

`OfType<Gift>()`:

EXAMPLE / FLOW

```
Keep compatible Gift values
Skip incompatible elements
```

`Cast<Gift>()`:

EXAMPLE / FLOW

```
Every element is expected
to be castable to Gift
```

36. SequenceEqual

C#

```
bool same = first.SequenceEqual(second);
```

Checks:

EXAMPLE / FLOW

```
Same number of elements

Same order

Equivalent elements
```

It is different from set equality.

37. IEnumerable<T> vs IQueryable<T>

This is the most important LINQ topic for backend developers.

IEnumerable<T>

Commonly means:

EXAMPLE / FLOW

```
LINQ operators execute
as .NET code
over objects
```

Example:

C#

```
IEnumerable<Gift> query = gifts.Where(gift => gift.TotalAmount > 1000m);
```

The predicate is executable .NET behavior.

IQueryable<T>

`IQueryable<T>` represents a query that can be interpreted by a provider.

Example:

C#

```
IQueryable<Gift> query = dbContext.Gifts.Where(gift => gift.TotalAmount > 1000m);
```

`Queryable` operators build expression trees describing the query rather than simply executing delegates over objects. The provider later interprets those expression trees, for example by translating them into SQL.

IEnumerable Flow

EXAMPLE / FLOW

```
List<Gift>
    ↓
Enumerable.Where
    ↓
Func<Gift,bool>
    ↓
C# code executes in process
```

IQueryable Flow

EXAMPLE / FLOW

```
DbSet<Gift>
    ↓
Queryable.Where
    ↓
Expression<Func<Gift,bool>>
    ↓
Expression tree
    ↓
EF Core provider
    ↓
SQL translation
    ↓
Database
```

Similar Syntax, Different Meaning

These look almost identical:

C#

```
IEnumerable<Gift> query = gifts.Where(gift => gift.TotalAmount > 1000m);
```

C#

```
IQueryable<Gift> query = dbContext.Gifts.Where(gift => gift.TotalAmount > 1000m);
```

But execution is fundamentally different.

LINQ to Objects

Lambda becomes:

EXAMPLE / FLOW

```
Executable delegate
```

IQueryable Provider

Lambda commonly becomes:

EXAMPLE / FLOW

```
Expression tree
```

that the provider attempts to translate.

Microsoft's LINQ overview explicitly notes that queries over `IEnumerable<T>` compile to delegates, while queries over `IQueryable<T>` can compile to expression trees for a provider.

38. Why Every C# Method Does Not Work in EF LINQ

Suppose:

C#

```
bool IsMajorGift(
    Gift gift)
{
    return gift.TotalAmount
```

```
        >
        10_000m;
}
```

LINQ / C#

Then:

```
C#
dbContext.Gifts
    .Where(
        gift =>
            IsMajorGift(
                gift))
```

may not be translatable.

The database provider doesn't have arbitrary knowledge of your C# method implementation.

Expression Must Be Translatable

This:

```
C#
gift =>
    gift.TotalAmount > 10_000m
```

can often translate naturally to SQL.

But:

```
C#
gift =>
    MyCustomCSharpMethod(
        gift)
```

may not.

EF Core attempts server evaluation wherever possible. If an expression outside the top-level projection cannot be translated, current EF Core generally throws rather than silently retrieving an entire table and filtering it on the client.

39. Server Evaluation

Ideal:

```
C#
List<GiftSummary> gifts =
    await dbContext.Gifts
        .Where(
            gift =>
                gift.TotalAmount >= 1000m)
        .Select(
            gift =>
                new GiftSummary(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToListAsync(
            cancellationToken);
```

Conceptually:

EXAMPLE / FLOW

```
Where
    ↓
translated to SQL

Select
    ↓
translated to SQL projection

Database returns only needed rows/columns
```

LINQ / C#

This is generally much better than loading entire entities first and filtering later.

EF Core recommends projecting only required properties and limiting result set size.

40. Explicit Client Evaluation

You can deliberately cross from provider-backed querying to .NET enumeration.

Example:

C#

```
IEnumerable<Gift> query =
    dbContext.Gifts
        .Where(
            gift =>
                gift.TotalAmount > 100m)
        .AsEnumerable()
        .Where(
            gift =>
                CustomDotNetRule(
                    gift));
```

The first filtering may be handled by the database.

After:

C#

```
AsEnumerable()
```

subsequent `Enumerable` operations execute in .NET as results are enumerated.

Microsoft describes `AsEnumerable` as an explicit way to switch to client-side LINQ while preserving streaming behavior instead of immediately buffering the results.

Important

`AsEnumerable()` by itself does **not** mean:

EXAMPLE / FLOW

```
Load everything into List immediately.
```

It changes which LINQ implementation subsequent calls bind to.

Materialization still happens when enumerated.

ToList Is Different

C#

```
List<Gift> gifts = dbContext.Gifts.Where(...).ToList();
```

means:

LINQ / C#

EXAMPLE / FLOW

```
Execute query
+
buffer results into memory
```

41. Materialization in EF Core

Common execution/materialization operations include:

EXAMPLE / FLOW

```
ToListAsync

ToArrayAsync

FirstAsync

SingleAsync

CountAsync

AnyAsync
```

EF Core builds a query expression until results are consumed by an execution operation.

Example

C#

```
IQueryable<Gift> query =
    dbContext.Gifts
        .Where(
            gift =>
                gift.TenantId
                == tenantId);
```

No SQL query is necessarily sent yet.

Then:

C#

```
List<Gift> results = await query.ToListAsync(cancellationToken);
```

Now the query executes.

42. Query Composition

One of the biggest benefits of deferred `IQueryable<T>` is composability.

C#

```
IQueryable<Gift> query =
    dbContext.Gifts
        .Where(
            gift =>
                gift.TenantId
                == tenantId);
```

Optional filter:

```C#
if (
    campaignId is not null)
{
    query = query.Where(gift => gift.CampaignId == campaignId);
}
```

Another:

```C#
if (
    minimumAmount is not null)
{
    query = query.Where(gift => gift.TotalAmount >= minimumAmount);
}
```

Then:

```C#
List<GiftSummary> results =
    await query
        .OrderByDescending(
            gift =>
                gift.GiftDate)
        .Select(
            gift =>
                new GiftSummary(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToListAsync(
            cancellationToken);
```

One final composed expression is sent to the provider.

43. Projection in EF Core

Avoid:

```C#
List<Gift> gifts = await dbContext.Gifts.ToListAsync();
```

if the API only needs:

```EXAMPLE / FLOW
GiftId

Amount
```

Prefer:

```C#
List<GiftSummary> gifts =
    await dbContext.Gifts
        .Select(
            gift =>
                new GiftSummary(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToListAsync();
```

This can reduce:

LINQ / C#

EXAMPLE / FLOW

```
Transferred columns

Entity creation

Change-tracking work

Memory usage
```

Microsoft explicitly recommends projecting only the properties you need.

44. Tracking vs No Tracking

EF Core entity queries are tracked by default.

For read-only queries:

C#

```
List<Gift> gifts =
    await dbContext.Gifts
        .AsNoTracking()
        .Where(
            gift =>
                gift.TenantId
                == tenantId).ToListAsync();
```

can avoid change-tracking overhead when updates aren't required.

Microsoft documents that tracking maintains entity state and identity resolution, while no-tracking queries can reduce tracking work for read-only scenarios.

Projection May Avoid Tracking Naturally

If you project only scalar/DTO data:

C#

```
var results =
    await dbContext.Gifts
        .Select(
            gift =>
                new
                {
                    gift.GiftId,
                    gift.TotalAmount
                })
        .ToListAsync();
```

and no entity instance is included in the result, EF Core has no returned entity instance to track.

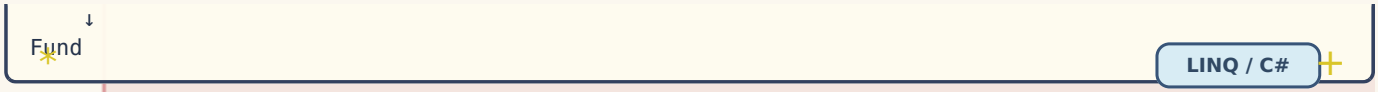
Microsoft documents this behavior for projections.

45. Related Data with Include

Suppose:

EXAMPLE / FLOW

```
Gift
    ↓
GiftAllocations
```



EF Core can load related data with:

```
C#
List<Gift> gifts =
    await dbContext.Gifts
        .Include(
            gift =>
                gift.Allocations)
        .ThenInclude(
            allocation =>
                allocation.Fund)
        .ToListAsync();
```

Microsoft documents `Include` and `ThenInclude` for eager loading related navigation data.

But Don't Include Everything Automatically

If the API needs:

```
EXAMPLE / FLOW
GiftId

FundName

AllocatedAmount
```

projection may be more efficient:

```
C#
var results =
    await dbContext.GiftAllocations
        .Select(
            allocation =>
                new
                {
                    allocation.GiftId,
                    FundName = allocation.Fund.Name, allocation.Amount }).ToListAsync();
```

A large graph of `Include` calls can retrieve far more information than the endpoint needs.

Collection Include Performance

Loading collection navigations can produce large joined result sets and, depending on the query shape, potentially lead to duplicated row data/cartesian expansion.

Microsoft cautions that collection eager loading can have performance implications and documents split-query alternatives where appropriate.

46. Buffering vs Streaming in EF Core

Buffering:

```
C#
List<Gift> gifts = await query.ToListAsync();
```

loads the entire result into memory.

Streaming:

LINQ / C#

```
C#
await foreach (
    Gift gift
    in query.AsAsyncEnumerable())
{
    Process(
        gift);
}
```

processes elements as they arrive instead of storing the entire result set first.

Microsoft distinguishes buffering through `ToList` / `ToArray` from streaming with asynchronous enumeration.

When Buffering Is Fine

If query returns:

```
EXAMPLE / FLOW
20 rows
```

a list is often simplest.

When Streaming Matters

If processing:

```
EXAMPLE / FLOW
2,000,000 rows
```

buffering everything can dramatically increase memory usage.

47. N+1 Query Problem

Consider:

```
C#
List<Gift> gifts = await dbContext.Gifts.ToListAsync();

foreach (
    Gift gift
    in gifts)
{
    foreach (
        GiftAllocation allocation
        in gift.Allocations)
    {
    }
}
```

If related data is loaded lazily and every Gift triggers another database query, you might execute:

```
EXAMPLE / FLOW
1 query for Gifts

+
N queries for Allocations
```

Hence:

EXAMPLE / FLOW

```
N + 1
```

Use intentional query design through:

EXAMPLE / FLOW

```
Projection

Include

Explicit joins

Correct loading strategy
```

rather than accidentally triggering repeated database queries.

48. LINQ Side Effects

Avoid this style:

C#

```
IEnumerable<Gift> query =
    gifts.Where(
        gift =>
        {
            Console.WriteLine(
                gift.GiftId);

            return gift.TotalAmount
                >
                100m;
        });
```

Because the query is deferred, logging occurs:

EXAMPLE / FLOW

```
When enumerated
```

and potentially:

EXAMPLE / FLOW

```
Every time enumerated
```

Worse

C#

```
int counter = 0;

IEnumerable<Gift> query =
    gifts.Select(
        gift =>
        {
            counter++;

            return gift;
        });
```

Now the number of side effects depends on enumeration.

LINQ works best when selectors and predicates behave like:


```
EXAMPLE / FLOW
Pure transformations / tests
```

rather than hidden imperative workflows.

49. Don't Use LINQ Just to Get Side Effects

Bad:

```
C#
gifts.Select(
    gift =>
    {
        SendReceipt(
            gift);

        return gift;
    })
    .ToList();
```

This abuses `Select`.

Better:

```
C#
foreach (
    Gift gift
    in gifts)
{
    SendReceipt(
        gift);
}
```

LINQ describes transformation/query operations.

A normal loop is often clearer for:

```
EXAMPLE / FLOW
Perform action for every element
```

50. foreach Is Not Obsolete

LINQ is powerful, but not every loop should become LINQ.

Use `foreach` when:

```
EXAMPLE / FLOW
Stateful processing is central

Multiple side effects happen

Control flow has break/continue complexity

Readability is better with statements
```

Use LINQ when:

EXAMPLE / FLOW

LINQ / C#

```
Filtering

Projection

Grouping

Aggregation

Ordering

Set operations
```

naturally describe the problem.

51. `let` in Query Syntax

Query syntax supports:

C#

```
let
```

to compute an intermediate value.

Example:

C#

```
var query =
    from gift in gifts
    let remaining =
        gift.TotalAmount
        -
        gift.Allocations.Sum(
            allocation =>
                allocation.Amount)
    where remaining > 0m
    select new
    {
        gift.GiftId,
        Remaining = remaining };
```

This can prevent repeating a complex expression.

52. `into` in Query Syntax

`into` continues a query after grouping or joining.

Example:

C#

```
var query =
    from gift in gifts
    group gift
        by gift.CampaignId
        into campaignGroup
    where campaignGroup.Count() > 5
    select campaignGroup;
```

`campaignGroup` becomes the new range variable.

53. Null Handling in LINQ

Example:

```C#
IEnumerable<string> emails =
    constituents
        .Where(
            constituent =>
                constituent.Email
                    is not null)
        .Select(
            constituent =>
                constituent.Email!);
```

Try to preserve good nullable typing so you don't need unnecessary:

```C#
!
```

suppression.

Projection with Safe Default

```C#
var result =
    constituents.Select(
        constituent =>
            new
            {
                constituent.ConstituentId,

                Email = constituent.Email ?? "Not provided" });
```

54. DefaultIfEmpty and Nullable Reference Types

Be careful because:

```C#
DefaultIfEmpty()
```

may introduce:

```C#
default(T)
```

For reference types:

EXAMPLE / FLOW

```
null
```

can appear.

Handle that explicitly where required.

55. Equality in LINQ

Operators such as:

EXAMPLE / FLOW

LINQ / C#

```
Distinct

Contains

Union

Intersect

Except

SequenceEqual
```

depend on equality.

For ordinary LINQ-to-Objects, this generally uses:

C#

```
EqualityComparer<T>.Default
```

or an explicitly supplied comparer.

Entity Equality Caveat

Suppose two different `Gift` instances contain:

EXAMPLE / FLOW

```
same GiftId
```

but the class uses reference equality.

Then:

C#

```
gifts.Distinct()
```

may still treat them as different objects.

If uniqueness should be by ID:

C#

```
gifts.DistinctBy(
    gift =>
        gift.GiftId)
```

is clearer.

56. Sorting Comparers

You can provide a comparer to:

C#

```
OrderBy
```

where supported.

Example:

```C#
IEnumerable<string> names =
    campaignNames.OrderBy(
        name =>
            name,
        StringComparer.OrdinalIgnoreCase);
```

Equality and ordering are separate concepts:

```EXAMPLE / FLOW
IEqualityComparer<T>
    →
same/different

IComparer<T>
    →
before/after
```

57. LINQ Performance: Use the Right Operator

If you need:

```EXAMPLE / FLOW
Does any Gift exist?
```

use:

```C#
Any()
```

not:

```C#
Count() > 0
```

Need First Match?

Use:

```C#
FirstOrDefault(...)
```

rather than:

```C#
Where(...)
    .ToList()[0]
```

Need Only Count?

Use:

```C#
Count(...)
```

rather than:

```
C#
Where(...)
    .ToList()
    .Count
```

The latter materializes values you didn't need.

Need Only Sum?

Use:

```
C#
Sum(...)
```

rather than first building a temporary list.

58. Filter Before Expensive Work

Better:

```
C#
gifts
    .Where(
        gift =>
            gift.Status
            == GiftStatus.Completed).Select(gift => ExpensiveProjection(gift));
```

than projecting every object first and filtering afterward, when semantics allow filtering earlier.

With database queries this can also allow less data/work to flow through later stages.

59. Avoid Accidental Multiple Enumeration

Bad:

```
C#
IEnumerable<Gift> query = GetRemoteOrExpensiveSequence();

if (
    query.Any())
{
    foreach (
        Gift gift
        in query)
    {
    }
}
```

This may enumerate twice.

Depending on the source:

```
EXAMPLE / FLOW
Fine

Expensive

Wrong

Multiple DB queries
```

```
One-time iterator problem
```

LINQ / C#

Possible Alternative

Materialize once if appropriate:

```C#
List<Gift> results = query.ToList();

if (
    results.Count > 0)
{
    foreach (
        Gift gift
        in results)
    {
    }
}
```

But do not materialize large datasets automatically.

60. Avoid `ToList()` in the Middle of a Database Query

Bad:

```C#
var results =
    dbContext.Gifts
        .Where(
            gift =>
                gift.TenantId
                ==
                tenantId)
        .ToList()
        .Where(
            gift =>
                gift.TotalAmount
                >= 1000m);
```

The second `Where` now runs:

EXAMPLE / FLOW

```
In memory
```

after the first query has already been materialized.

Better:

```C#
var results =
    await dbContext.Gifts
        .Where(
            gift =>
                gift.TenantId
                ==
                tenantId)
        .Where(
            gift =>
```

```
            gift.TotalAmount
            >= 1000m).ToListAsync();
```

LINQ / C#

61. Projection Before Materialization

Good:

C#

```
List<GiftSummary> results =
    await dbContext.Gifts
        .Where(
            gift =>
                gift.TenantId
                ==
                tenantId)
        .Select(
            gift =>
                new GiftSummary(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToListAsync();
```

Flow:

EXAMPLE / FLOW

```
Compose
    ↓
Translate
    ↓
Database filters
    ↓
Database selects columns
    ↓
Materialize final DTOs
```

62. Dynamic Sorting Security

Suppose the API accepts:

EXAMPLE / FLOW

```
sortBy = "Amount"
```

Do not blindly turn arbitrary user input into unrestricted expression/reflection execution.

Whitelist supported fields:

C#

```
query =
    sortBy switch
    {
        "amount"
            =>
            query.OrderBy(
                gift =>
                    gift.TotalAmount),

        "date"
            =>
            query.OrderBy(
                gift =>
                    gift.GiftDate),
```

```
*       _
            =>
            query.OrderBy(
                gift =>
                    gift.GiftId)
    };
```

LINQ / C#

EF Core parameterizes normal query values, but application code must still validate untrusted input and ensure dynamic query construction only permits intended expressions.

63. Tenant Filtering

For a multi-tenant system, this is dangerous:

C#

```
IQueryable<Gift> query = dbContext.Gifts;
```

followed by user-controlled filtering without guaranteeing Tenant scope.

A stronger default:

C#

```
IQueryable<Gift> query =
    dbContext.Gifts
        .Where(
            gift =>
                gift.TenantId
                == tenantId);
```

Then compose further filters onto the already tenant-scoped query.

LINQ gives you query composition.

It does not automatically provide:

EXAMPLE / FLOW

```
Authorization

Tenant isolation

Access control
```

64. Unbounded Queries

Bad API:

C#

```
return await dbContext.Gifts
    .ToListAsync();
```

when the table may contain:

EXAMPLE / FLOW

```
Millions of rows
```

Potential problems:

EXAMPLE / FLOW

```
Database load

Network transfer

Memory pressure

Serialization time

Slow responses
```

LINQ / C#

Use:

EXAMPLE / FLOW

```
Filters

Pagination

Result limits

Projection
```

where appropriate.

EF Core performance guidance explicitly recommends limiting result-set size.

65. Async LINQ with EF Core

Database I/O should normally use async materialization operators:

C#

```
await query.ToListAsync(
    cancellationToken);
```

C#

```
await query.FirstOrDefaultAsync(
    cancellationToken);
```

C#

```
await query.AnyAsync(
    cancellationToken);
```

C#

```
await query.CountAsync(
    cancellationToken);
```

This allows database I/O to use asynchronous APIs rather than blocking request threads.

EF Core recommends asynchronous query APIs for scalable application I/O.

66. LINQ Is Not Automatically Async

This:

C#

```
gifts.Where(...)
```

is not asynchronous.

LINQ-to-Objects normally runs synchronously.

Database-provider execution may expose async terminal operations such as:

```
C#
ToListAsync
```

because actual I/O occurs during query execution.

67. Query Object vs Results

This distinction must become automatic.

```
C#
IQueryable<Gift> query = dbContext.Gifts.Where(...);
```

means:

```
EXAMPLE / FLOW
Query description
```

not:

```
EXAMPLE / FLOW
List of Gifts
```

Results

```
C#
List<Gift> results = await query.ToListAsync();
```

Now you have:

```
EXAMPLE / FLOW
Materialized objects
```

68. Returning IQueryable from Layers

Technically you can write:

```
C#
public IQueryable<Gift>
    GetGifts()
{
    return _dbContext.Gifts;
}
```

But doing so leaks query composition and provider behavior to callers.

That may expose:

```
EXAMPLE / FLOW
Persistence details

Lifetime issues

Unexpected queries
```

```
Security-filter bypass possibilities
```

LINQ / C#

Whether repositories should expose `IQueryable<T>` is an architecture decision, not a LINQ requirement.

We'll evaluate that in Topic 35.

69. Common Mistake — Thinking Where Creates a List

This:

```C#
var filtered = gifts.Where(...);
```

usually produces:

```EXAMPLE / FLOW
Deferred IEnumerable<T>
```

not:

```EXAMPLE / FLOW
List<T>
```

If you need a list:

```C#
List<Gift> filtered = gifts.Where(...).ToList();
```

Common Mistake — Assuming Query Runs Immediately

```C#
var query = gifts.Where(gift => gift.TotalAmount > 100m);
```

may do no actual filtering yet.

It creates the query recipe.

Common Mistake — Modifying Source Before Deferred Enumeration

```C#
var query = gifts.Where(gift => gift.TotalAmount > 100m);

gifts.Add(
    new Gift(...));
```

The new element may appear when the query is later enumerated.

If you need snapshot semantics:

```C#
List<Gift> snapshot = query.ToList();
```

Common Mistake — Multiple Enumeration of Database Query

```C#
if (
    await query.AnyAsync())
{
    List<Gift> gifts = await query.ToListAsync();
}
```

This performs:

```EXAMPLE / FLOW
Existence query

Then data query
```

Sometimes intentional.

Sometimes unnecessary.

If you need the actual values anyway, simply retrieving the page may be enough.

Common Mistake — First When You Mean Single

If database rules say:

```EXAMPLE / FLOW
ReferenceNumber is globally unique
```

then:

```C#
FirstOrDefault(...)
```

can hide duplicate-data bugs.

`SingleOrDefault` can communicate and verify the uniqueness assumption in memory/provider semantics.

But uniqueness should primarily be enforced by:

```EXAMPLE / FLOW
Database unique constraint/index
```

not only by LINQ.

Common Mistake — Single When Multiple Is Legitimate

If multiple Gifts can belong to a Campaign:

```C#
Single(
    gift =>
        gift.CampaignId
        ==
        campaignId)
```

is wrong.

Use:

```C#
Where(...)
```

or another operation matching the relationship.

Common Mistake — Count > 0

Prefer:

```C#
Any()
```

for existence.

Common Mistake — Select for Side Effects

Avoid:

```C#
gifts.Select(
    gift =>
    {
        SendReceipt(
            gift);

        return gift;
    })
    .ToList();
```

Use:

```C#
foreach
```

for imperative side-effect workflows.

Common Mistake — Assuming Query Syntax Is SQL

This:

```C#
from gift in query
where ...
select ...
```

looks SQL-like.

But it is:

```EXAMPLE / FLOW
C# syntax
```

compiled to LINQ method calls.

With LINQ-to-Objects, no SQL is involved.

Common Mistake — Assuming IQueryable Means Database

`IQueryable<T>` is a provider abstraction.

A provider might target:

EXAMPLE / FLOW

```
Database

Remote API

Custom query engine

Other data source
```

Entity Framework Core is a common provider, but `IQueryable<T>` itself is not synonymous with SQL Server.

Common Mistake — Arbitrary C# in EF Query

A lambda that works perfectly with:

C#

```
List<Gift>
```

may not be translatable by:

C#

```
DbSet<Gift>
```

because LINQ-to-Objects executes C# code, whereas EF must translate its expression tree.

Common Mistake — AsEnumerable Too Early

C#

```
dbContext.Gifts
    .AsEnumerable()
    .Where(
        gift =>
            gift.TenantId
            == tenantId);
```

moves the Tenant filter into client-side LINQ.

Potentially the database may return far more rows than necessary.

Keep translatable filtering on the provider side.

Common Mistake — ToList Too Early

C#

```
dbContext.Gifts
    .ToList()
    .Where(...)
```

materializes too soon.

Compose first.

Materialize last.

Common Mistake — Projecting Entire Entity When Only Two Fields Needed

Bad:

```C#
List<Gift> gifts = await dbContext.Gifts.AsNoTracking().ToListAsync();
```

when endpoint only needs:

```EXAMPLE / FLOW
GiftId

Amount
```

Prefer DTO projection.

Common Mistake — Unordered Paging

Bad:

```C#
query
    .Skip(
        100)
    .Take(
        50);
```

without deterministic ordering.

Paging should have a stable order.

Common Mistake — Deep Offset Pagination Everywhere

Large:

```C#
Skip(5_000_000)
```

can be expensive.

Consider keyset pagination for large sequential navigation scenarios.

Common Mistake — Large ToList for Streaming Work

If processing one item at a time from a huge dataset:

```C#
ToListAsync()
```

buffers everything.

Streaming may be better.

Common Mistake — Returning IQueryable Beyond DbContext Lifetime

LINQ / C#

If:

```
C#
DbContext
```

is disposed before the returned query is enumerated, execution can fail.

Remember:

```
EXAMPLE / FLOW
IQueryable query
```

may not have executed yet.

Common Mistake — Confusing Select and SelectMany

`Select` can create nested sequences.

`SelectMany` flattens them.

For:

```
EXAMPLE / FLOW
Gift → Allocations
```

use:

```
C#
SelectMany(
    gift =>
        gift.Allocations)
```

when you need one flat allocation stream.

Common Mistake — GroupBy for Everything

If you only need frequency counts:

```
C#
CountBy
```

may be clearer and can avoid intermediate grouping objects in LINQ-to-Objects.

Best Practices

Use LINQ when the problem is naturally about:

```
EXAMPLE / FLOW
Filtering

Projection

Sorting

Grouping
```

```
Joining
*
Aggregation

Set operations
```

Prefer method syntax for most general-purpose application code, but use query syntax where it significantly improves readability.

Remember:

EXAMPLE / FLOW

```
Query definition
≠
Query execution
```

Know whether an operation is:

EXAMPLE / FLOW

```
Deferred

Immediate

Streaming

Non-streaming
```

Use:

C#

```
Where
```

for filtering.

Use:

C#

```
Select
```

for one-to-one projection.

Use:

C#

```
SelectMany
```

for flattening nested sequences.

Use:

C#

```
OrderBy
```

for primary ordering and:

C#

```
ThenBy
```

for secondary ordering.

Use:

```C#
Any
```

for existence.

Use:

```C#
All
```

for universal conditions.

Use:

```C#
FirstOrDefault
```

when zero or many are allowed but you need the first.

Use:

```C#
SingleOrDefault
```

when zero or exactly one is the valid cardinality.

Use:

```C#
GroupBy
```

when you actually need groups.

Consider:

```C#
CountBy
```

for frequency counting and:

```C#
AggregateBy
```

for direct per-key accumulation in modern .NET.

Use:

```C#
DistinctBy
```

when uniqueness depends on a key.

Keep equality rules explicit.

Avoid side effects inside:

EXAMPLE / FLOW

```
Where

Select
```

```
OrderBy key selectors
```

unless there is a very deliberate reason.

Do not assume LINQ methods mutate the original sequence.

Materialize with:

```C#
ToList

ToArray
```

only when you actually need stored results.

Watch for multiple enumeration.

With EF Core:

```EXAMPLE / FLOW
Compose first

Materialize last
```

Keep filtering and projection provider-side whenever possible.

Project only needed columns.

Limit result set size.

Use deterministic ordering for pagination.

Consider keyset pagination for large sequential datasets.

Use:

```C#
AsNoTracking()
```

for read-only entity queries when change tracking is unnecessary.

Do not call:

```C#
AsEnumerable()
```

too early.

Understand that:

```EXAMPLE / FLOW
IEnumerable<T>
    →
delegates / in-process execution

IQueryable<T>
    →
expression trees / provider execution
```

Do not assume arbitrary .NET methods translate to SQL.

Validate all externally supplied filtering, sorting, paging, and dynamic query inputs.

Do not use LINQ as a security boundary.

Practical Exercise 1 — Filter Gifts

LINQ / C#

Given:

```C#
List<Gift> gifts;
```

return completed gifts above $1,000:

```C#
IEnumerable<Gift> result =
    gifts.Where(
        gift =>
            gift.Status
                == GiftStatus.Completed && gift.TotalAmount >= 1000m);
```

Practical Exercise 2 — Project DTOs

```C#
IEnumerable<GiftSummary> result =
    gifts.Select(
        gift =>
            new GiftSummary(
                gift.GiftId,
                gift.TotalAmount));
```

Practical Exercise 3 — Sort by Amount

```C#
IEnumerable<Gift> result =
    gifts
        .OrderByDescending(
            gift =>
                gift.TotalAmount)
        .ThenBy(
            gift =>
                gift.GiftId);
```

Practical Exercise 4 — Flatten Allocations

```C#
IEnumerable<GiftAllocation> allocations = gifts.SelectMany(gift => gift.Allocations);
```

Practical Exercise 5 — Group by Campaign

```C#
var totals =
    gifts
        .GroupBy(
            gift =>
                gift.CampaignId)
        .Select(
            group =>
                new
                {
                    CampaignId =
```

```
                    group.Key,
                Count =
                    group.Count(),

                Total = group.Sum(gift => gift.TotalAmount) });
```

LINQ / C#

Practical Exercise 6 — Remove Duplicate Fund IDs

C#

```
IEnumerable<long> unique = fundIds.Distinct();
```

Or by allocation object:

C#

```
IEnumerable<GiftAllocation>
    unique = allocations.DistinctBy(allocation => allocation.FundId);
```

Practical Exercise 7 — Existence Check

Bad:

C#

```
if (
    gifts.Count(
        gift =>
            gift.Status
            ==
            GiftStatus.Pending)
    >
    0)
{
}
```

Better:

C#

```
if (
    gifts.Any(
        gift =>
            gift.Status
            ==
            GiftStatus.Pending))
{
}
```

Practical Exercise 8 — Exactly One Gift

C#

```
Gift gift = gifts.Single(gift => gift.GiftId == giftId);
```

Use only if:

EXAMPLE / FLOW

```
Exactly one
```

is a required invariant.

Practical Exercise 9 — Deferred Execution

LINQ / C#

```C#
List<int> numbers = [1, 2, 3];

IEnumerable<int> query = numbers.Where(value => value > 1);

numbers.Add(
    4);
```

Then:

```C#
foreach (
    int value
    in query)
{
    Console.WriteLine(
        value);
}
```

Output:

EXAMPLE / FLOW

```
2
3
4
```

because enumeration happens after `4` is added.

Practical Exercise 10 — Database Projection

Instead of:

```C#
List<Gift> gifts = await dbContext.Gifts.AsNoTracking().ToListAsync();
```

for a summary endpoint:

```C#
List<GiftSummary> summaries =
    await dbContext.Gifts
        .AsNoTracking()
        .Where(
            gift =>
                gift.TenantId
                ==
                tenantId)
        .Select(
            gift =>
                new GiftSummary(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToListAsync(
            cancellationToken);
```

Practical Exercise 11 — Pagination

C#

```
List<GiftSummary> page =
    await dbContext.Gifts
        .Where(
            gift =>
                gift.TenantId
                ==
                tenantId)
        .OrderByDescending(
            gift =>
                gift.GiftDate)
        .ThenByDescending(
            gift =>
                gift.GiftId)
        .Skip(
            (pageNumber - 1)
            *
            pageSize)
        .Take(
            pageSize)
        .Select(
            gift =>
                new GiftSummary(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToListAsync(
            cancellationToken);
```

Practical Exercise 12 — Modern CountBy

C#

```
var counts = gifts.CountBy(gift => gift.Status);

foreach (
    (GiftStatus status, int count)
    in counts)
{
    Console.WriteLine(
        $"{status}: {count}");
}
```

Exactly 25 Interview Questions

1. What is LINQ in C#?

LINQ stands for Language Integrated Query.

It provides a consistent C# model for querying and transforming data through operations such as:

EXAMPLE / FLOW

```
Where

Select

OrderBy

GroupBy
```

```
Join
*
Sum

Any
```

LINQ / C#

It works directly over `IEnumerable<T>` and can also work through providers implementing `IQueryable<T>`.

2. What is the difference between query syntax and method syntax?

Query syntax:

```C#
var query = from gift in gifts where gift.TotalAmount > 1000m select gift;
```

Method syntax:

```C#
var query = gifts.Where(gift => gift.TotalAmount > 1000m);
```

The compiler translates query syntax into standard query-operator method calls. Some operations are available only through method syntax.

3. What is deferred execution?

Deferred execution means a query is defined now but evaluated later when the sequence is enumerated.

Example:

```C#
var query = gifts.Where(...);
```

doesn't necessarily process the source immediately.

Execution commonly occurs during:

```C#
foreach
```

or another terminal/materializing operation.

4. What is immediate execution?

An operator executes immediately when it must produce its answer before returning.

Examples include:

EXAMPLE / FLOW

```
Count

Any

First

Single

Sum

Average

Min
```

```
Max
```

LINQ / C#

Materializers such as `ToList` and `ToArray` also force execution.

5. What is the difference between streaming and non-streaming LINQ operators?

A streaming operator such as:

```
C#
Where
```

can usually process and yield items incrementally.

A non-streaming operator such as:

```
C#
OrderBy
```

typically needs to read the source before it can produce its first ordered result.

Microsoft distinguishes these execution categories for deferred LINQ operators.

6. What is the difference between `Where` and `Select`?

`Where`:

```
EXAMPLE / FLOW
Filters elements
```

and normally preserves the element type.

`Select`:

```
EXAMPLE / FLOW
Transforms/projects elements
```

and may change the result type.

Example:

```
C#
gifts.Where(
    gift =>
        gift.TotalAmount > 1000m);
```

versus:

```
C#
gifts.Select(
    gift =>
        gift.TotalAmount);
```

7. What is `SelectMany`?

`SelectMany` projects each source element to a child sequence and flattens all child sequences into one result sequence.

For:

EXAMPLE / FLOW

```
Gift
    →
GiftAllocations
```

LINQ / C#

use:

C#

```
gifts.SelectMany(
    gift =>
        gift.Allocations);
```

to obtain:

EXAMPLE / FLOW

```
IEnumerable<GiftAllocation>
```

instead of a sequence of allocation collections.

8. What is the difference between `OrderBy` and `ThenBy`?

`OrderBy` starts a primary ordering.

`ThenBy` adds a secondary ordering to an existing ordered sequence.

Example:

C#

```
gifts
    .OrderBy(
        gift =>
            gift.Status)
    .ThenByDescending(
        gift =>
            gift.TotalAmount);
```

Calling another `OrderBy` would replace the previous primary sort relationship rather than adding a secondary key.

9. What is the difference between `Any` and `Count() > 0`?

`Any` directly expresses an existence test and can stop once one element is found.

`Count()` is designed to determine a count and may require examining more data.

For existence checks, prefer:

C#

```
query.Any()
```

10. What is the difference between `First`, `FirstOrDefault`, `Single`, and `SingleOrDefault`?

EXAMPLE / FLOW

```
First
    →
first item,
throws if none

FirstOrDefault
    →
```

```
first item,
default if none

Single
    →
exactly one required,
throws for zero or multiple

SingleOrDefault
    →
zero or one allowed,
throws if multiple
```

LINQ / C#

Choose based on the actual cardinality rule.

11. What is GroupBy?

`GroupBy` partitions elements according to a key.

Example:

C#

```
gifts.GroupBy(
    gift =>
        gift.CampaignId);
```

produces groups where each group exposes:

EXAMPLE / FLOW

```
Key
+
matching Gifts
```

12. What is the difference between GroupBy and ToLookup?

`GroupBy` returns a deferred grouped query.

`ToLookup` immediately materializes a reusable key-to-multiple-values lookup structure.

Use `ToLookup` when you deliberately want an in-memory grouped lookup that can repeatedly be indexed by key.

13. What is the difference between Select and SelectMany?

`Select` preserves one result per source element.

`SelectMany` flattens nested child sequences.

EXAMPLE / FLOW

```
Select:
Gift → IReadOnlyList<GiftAllocation>

SelectMany:
Gift → each GiftAllocation
```

14. What is the difference between Distinct and DistinctBy?

`Distinct` determines uniqueness using the element's equality semantics.

`DistinctBy` determines uniqueness using a selected key.

Example:

C#

```
gifts.DistinctBy(
    gift =>
        gift.ConstituentId);
```

LINQ / C#

keeps one element per selected constituent key.

15. What is the difference between `Concat` and `Union`?

`Concat` appends two sequences and preserves duplicates.

`Union` performs set union and returns unique elements according to equality semantics.

16. What is the difference between `Join` and `GroupJoin`?

`Join` performs an inner equijoin and produces one result for each matching pair.

`GroupJoin` associates each outer element with a collection of matching inner elements.

Microsoft describes `GroupJoin` as a grouped/hierarchical correlation that can participate in both inner and outer join patterns.

17. What changed for outer joins in .NET 10?

.NET 10 added standard:

C#

```
LeftJoin

RightJoin
```

operators to `Enumerable` and `Queryable`.

Before this, left joins were commonly represented with:

EXAMPLE / FLOW

```
GroupJoin
+
DefaultIfEmpty
+
SelectMany
```

patterns.

18. What is the difference between `IEnumerable<T>` and `IQueryable<T>`?

`IEnumerable<T>` generally represents in-process enumeration where LINQ-to-Objects operators execute .NET delegates.

`IQueryable<T>` represents a provider-backed query where operators build expression trees that the provider can interpret or translate.

With EF Core:

EXAMPLE / FLOW

```
IQueryable<T>
    ↓
Expression tree
    ↓
SQL translation
    ↓
Database
```

19. Why can a LINQ expression work on List but fail with EF Core?

LINQ / C#

`List<T>` uses LINQ-to-Objects, so your lambda executes as normal .NET code.

EF Core must translate the expression tree into SQL or another provider-supported form.

An arbitrary custom C# method may have no SQL equivalent.

Current EF Core generally throws if a non-translatable expression appears outside the allowed top-level client projection.

20. What does `AsEnumerable()` do to an EF Core query?

It switches subsequent LINQ operators to `Enumerable` rather than `Queryable`.

That means later operations execute as client-side .NET LINQ when enumeration occurs.

It does not itself create a `List<T>` or buffer every result.

21. Why should `ToList()` usually appear near the end of an EF Core query?

Calling `ToList` materializes the query.

Operators after that point run against in-memory objects rather than remaining part of the database expression.

Therefore:

EXAMPLE / FLOW

```
Compose filters/projections first

Materialize last
```

usually allows the database to do more of the filtering and projection efficiently.

22. What is the difference between buffering and streaming?

Buffering:

C#

```
ToListAsync()
```

loads all results into an in-memory collection.

Streaming:

C#

```
AsAsyncEnumerable()
```

can process results incrementally.

Streaming can reduce memory usage for large result sets.

23. What are `CountBy` and `AggregateBy`?

They are modern LINQ operators added in .NET 9.

`CountBy` counts elements per key.

`AggregateBy` performs a custom accumulation independently for each key.

They can avoid building intermediate `GroupBy` groupings for some frequency/aggregation scenarios.

24. What are common LINQ performance mistakes with EF Core?

Common mistakes include:

EXAMPLE / FLOW

```
Calling ToList too early

Loading entire entities when only a few columns are required

Returning huge unbounded result sets

Repeatedly enumerating database queries

Calling AsEnumerable before server-side filters

Loading too many related entities

Using deep offset pagination unnecessarily

Performing avoidable tracking for read-only queries
```

EF Core recommends projection, result limiting, careful tracking choices, and conscious buffering/streaming.

25. What is the most important LINQ rule for backend development?

Know **where the query executes**.

The same-looking LINQ expression may represent:

EXAMPLE / FLOW

```
In-memory .NET iteration
```

or:

EXAMPLE / FLOW

```
A provider-translated database query
```

Therefore always understand:

EXAMPLE / FLOW

```
Source type

Deferred execution

Materialization boundary

Translation capability

Result size

Number of enumerations
```

before judging correctness or performance.

Quick Revision

Concept	Meaning
LINQ	Language-integrated querying

Concept	Meaning
`Where`	Filter
`Select`	Project/transform
`SelectMany`	Flatten
`OrderBy`	Primary ascending order
`ThenBy`	Secondary order
`GroupBy`	Group by key
`Join`	Inner equijoin
`GroupJoin`	Grouped correlation
`LeftJoin`	.NET 10 left outer join
`RightJoin`	.NET 10 right outer join
`Any`	At least one?
`All`	Do all match?
`First`	First or exception
`Single`	Exactly one
`Sum`	Numeric sum
`Count`	Number of elements
`MaxBy`	Element with largest key
`Distinct`	Unique elements
`DistinctBy`	Unique by selected key
`Union`	Set union
`Intersect`	Set intersection
`Except`	Set difference
`Take`	Keep first N
`Skip`	Skip first N
`ToList`	Execute and buffer
Deferred execution	Query executes later
Streaming	Incremental processing
`IEnumerable<T>`	In-process enumeration
`IQueryable<T>`	Provider expression query
`AsEnumerable`	Switch to LINQ-to-Objects
`AsNoTracking`	EF read-only tracking optimization
`CountBy`	Count elements by key
`AggregateBy`	Aggregate independently by key

Complete Fundraising Example

Suppose an API must return:

EXAMPLE / FLOW

```
Completed Gifts

For one Tenant

Optionally one Campaign

At least a requested minimum amount

Newest first

Paged

Projected to DTOs
```

DTO

C#

```
public sealed record GiftSearchResult(
    long GiftId,
    long ConstituentId,
    long? CampaignId,
    decimal TotalAmount,
    DateTimeOffset GiftDate);
```

Search Method

C#

```
public async Task<IReadOnlyList<GiftSearchResult>>
    SearchAsync(
        long tenantId,
        long? campaignId,
        decimal? minimumAmount,
        int pageNumber,
        int pageSize,
        CancellationToken cancellationToken)
{
    if (
        tenantId <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(tenantId));
    }

    if (
        pageNumber <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(pageNumber));
    }

    if (
        pageSize
        is <= 0
        or > 200)
    {
```

```
        throw new ArgumentOutOfRangeException(
            nameof(pageSize));
    }

    IQueryable<Gift> query =
        _dbContext.Gifts
            .AsNoTracking()
            .Where(
                gift =>
                    gift.TenantId
                        == tenantId && gift.Status == GiftStatus.Completed);

    if (
        campaignId is not null)
    {
        query = query.Where(gift => gift.CampaignId == campaignId);
    }

    if (
        minimumAmount is not null)
    {
        query = query.Where(gift => gift.TotalAmount >= minimumAmount);
    }

    return await query
        .OrderByDescending(
            gift =>
                gift.GiftDate)
        .ThenByDescending(
            gift =>
                gift.GiftId)
        .Skip(
            (pageNumber - 1)
            *
            pageSize)
        .Take(
            pageSize)
        .Select(
            gift =>
                new GiftSearchResult(
                    gift.GiftId,
                    gift.ConstituentId,
                    gift.CampaignId,
                    gift.TotalAmount,
                    gift.GiftDate))
        .ToListAsync(
            cancellationToken);
}
```

LINQ / C#

What Happens Step by Step

Start:

EXAMPLE / FLOW

```
DbSet<Gift>
    ↓
IQueryable<Gift>
```

Tenant and status filter:

EXAMPLE / FLOW

```
WHERE TenantId = ...
AND Status = ...
```

LINQ / C#

Optional Campaign filter:

EXAMPLE / FLOW

```
AND CampaignId = ...
```

Optional amount filter:

EXAMPLE / FLOW

```
AND TotalAmount >= ...
```

Ordering:

EXAMPLE / FLOW

```
ORDER BY GiftDate DESC,
         GiftId DESC
```

Pagination:

EXAMPLE / FLOW

```
Skip / Take
```

Projection:

EXAMPLE / FLOW

```
Only DTO fields requested
```

Execution:

EXAMPLE / FLOW

```
ToListAsync
```

Before the final execution:

EXAMPLE / FLOW

```
The query is being composed.
```

At:

C#

```
ToListAsync()
```

EF Core executes the provider query and materializes results.

Why Tenant Filter Comes First Conceptually

The most important business/security scope is:

EXAMPLE / FLOW

```
Tenant
```

So the query begins from:

C#

```
.Where(
    gift =>
        gift.TenantId
        ==
        tenantId)
```

LINQ / C#

Then optional client-request filters narrow the already tenant-scoped query.

LINQ helps compose those conditions, but security still comes from intentional application design.

Fundraising Reporting Example

Find total completed gift amount by Campaign:

C#

```
var campaignTotals =
    await _dbContext.Gifts
        .AsNoTracking()
        .Where(
            gift =>
                gift.TenantId
                    ==
                    tenantId
                &&
                gift.Status
                    ==
                    GiftStatus.Completed)
        .GroupBy(
            gift =>
                gift.CampaignId)
        .Select(
            group =>
                new
                {
                    CampaignId =
                        group.Key,

                    GiftCount =
                        group.Count(),

                    TotalAmount =
                        group.Sum(
                            gift =>
                                gift.TotalAmount),

                    AverageAmount =
                        group.Average(
                            gift =>
                                gift.TotalAmount)
                })
        .ToListAsync(
            cancellationToken);
```

Flow:

EXAMPLE / FLOW

```
Tenant Gifts
    ↓
Completed only
    ↓
Group by Campaign
```

```
    ↓
Count
Sum
Average
    ↓
Return summary rows
```

LINQ / C#

Fundraising Allocation Example

Flatten all allocations:

C#

```
IEnumerable<GiftAllocation> allocations = gifts.SelectMany(gift => gift.Allocations);
```

Funds Used by Gifts

C#

```
IEnumerable<long> uniqueFundIds =
    gifts
        .SelectMany(
            gift =>
                gift.Allocations)
        .Select(
            allocation =>
                allocation.FundId)
        .Distinct();
```

Flow:

EXAMPLE / FLOW

```
Gifts
    ↓
All allocations
    ↓
Fund IDs
    ↓
Distinct
```

Constituents with Large Gifts

C#

```
var majorDonorIds =
    gifts
        .Where(
            gift =>
                gift.TotalAmount
                >= 10_000m).Select(gift => gift.ConstituentId).Distinct();
```

Campaign Frequency with CountBy

In memory:

C#

```
var counts = gifts.CountBy(gift => gift.CampaignId);
```

Result:

EXAMPLE / FLOW

```
Campaign 100 → 14 Gifts

Campaign 200 → 8 Gifts

Campaign 300 → 25 Gifts
```

LINQ / C#

Allocated Amount by Fund

C#

```
var totalsByFund =
    gifts
        .SelectMany(
            gift =>
                gift.Allocations)
        .GroupBy(
            allocation =>
                allocation.FundId)
        .Select(
            group =>
                new
                {
                    FundId =
                        group.Key,

                    Total = group.Sum(allocation => allocation.Amount) });
```

Joining Gift and Campaign

C#

```
var result =
    gifts.Join(
        campaigns,
        gift =>
            gift.CampaignId,
        campaign =>
            campaign.CampaignId,
        (
            gift,
            campaign) =>
            new
            {
                gift.GiftId,
                campaign.Name,
                gift.TotalAmount
            });
```

Complete LINQ Decision Flow

EXAMPLE / FLOW

```
Have a sequence?
    ↓
Need to remove unwanted elements?
    →
Where
```

EXAMPLE / FLOW

Need different result shape?

→

Select

EXAMPLE / FLOW

Need to flatten child collections?

→

SelectMany

EXAMPLE / FLOW

Need order?

→

OrderBy / ThenBy

EXAMPLE / FLOW

Need groups?

→

GroupBy

EXAMPLE / FLOW

Need frequency counts?

→

CountBy

EXAMPLE / FLOW

Need unique elements?

→

Distinct / DistinctBy

EXAMPLE / FLOW

Need two sequences matched by key?

→

Join / GroupJoin / LeftJoin / RightJoin

EXAMPLE / FLOW

Need first item?

→

First / FirstOrDefault

EXAMPLE / FLOW

Need exactly one?

→

Single / SingleOrDefault

EXAMPLE / FLOW

Need existence?

→

Any

EXAMPLE / FLOW

```
Need numeric summary?
    →
Sum / Average / Min / Max
```

EXAMPLE / FLOW

```
Need stored results?
    →
ToList / ToArray
```

LINQ-to-Objects Execution Flow

EXAMPLE / FLOW

```
List<Gift>
    ↓
Where
    ↓
Func<Gift,bool>
    ↓
Select
    ↓
Func<Gift,TResult>
    ↓
Enumeration
    ↓
C# executes locally
```

EF Core LINQ Execution Flow

EXAMPLE / FLOW

```
DbSet<Gift>
    ↓
IQueryable<Gift>
    ↓
Where / Select / OrderBy
    ↓
Expression tree grows
    ↓
ToListAsync
    ↓
EF Core provider
    ↓
SQL translation
    ↓
SQL Server
    ↓
Rows returned
    ↓
Objects / DTOs materialized
```

This is the central backend LINQ mental model.

Performance Mental Model

EXAMPLE / FLOW

```
Filter early
    ↓
Project only needed columns
```

```
    ↓
Limit rows
    ↓
Order deliberately
    ↓
Materialize late
```

Avoid:

EXAMPLE / FLOW

```
Load everything
    ↓
ToList
    ↓
Filter in memory
```

when the database could perform the filtering.

Final End-to-End Mental Model

EXAMPLE / FLOW

```
Source
    ↓
IEnumerable<T>?
    ├─ Yes
    │    ↓
    │ LINQ-to-Objects
    │    ↓
    │ delegates execute in .NET
    │
    └─ IQueryable<T>?
         ↓
       Expression tree
         ↓
       Query provider
         ↓
       SQL / remote query
```

Then:

EXAMPLE / FLOW

```
Where
    →
filter
```

EXAMPLE / FLOW

```
Select
    →
project
```

EXAMPLE / FLOW

```
SelectMany
    →
flatten
```

EXAMPLE / FLOW

```
OrderBy / ThenBy
    →
sort
```

EXAMPLE / FLOW

```
GroupBy
    →
group
```

EXAMPLE / FLOW

```
Join
    →
correlate
```

EXAMPLE / FLOW

```
Any
    →
existence
```

EXAMPLE / FLOW

```
Single
    →
uniqueness expectation
```

EXAMPLE / FLOW

```
Sum / Count / Average
    →
aggregate
```

EXAMPLE / FLOW

```
ToList
    →
execute + buffer
```

The most important lesson is:

> **LINQ is not simply shorthand for `foreach`. It is a composable query model. With `IEnumerable<T>`, LINQ executes normal .NET delegates over objects. With `IQueryable<T>`, the same-looking operations build expression trees for a provider such as EF Core to translate. Correct LINQ therefore requires understanding deferred execution, materialization, cardinality, provider translation, result size, and where each part of the query actually runs.**

For the Fundraising Platform, a strong default database-query pipeline is:

EXAMPLE / FLOW

```
Start with tenant-scoped IQueryable
        ↓
Apply business/search filters
        ↓
Apply deterministic ordering
        ↓
Apply pagination
        ↓
```

```
Project directly to response DTO
*       ↓
Materialize asynchronously
```

LINQ / C#

That usually produces code that is:

EXAMPLE / FLOW

```
Readable

Composable

Type-safe

Efficient

Database-friendly
```

when the query is designed carefully.

Next roadmap topic: Topic 25 — Extension Methods, Partial Types, and Anonymous Types.